to what condition would make the test fail, not what condition would make the test pass.

Here is a test that is not straightforward:

```
## Don't do this
it "should add to 37" do
  expect(User.all_total_points).to eq(37)
end
```

Where does the 37 come from? It's part of the global setup. If you were to peek into this fake example's user fixture file, you'd see that somehow the totals of the points of all the users in that file add up to 37. The test passes. Yay?

There are two relevant problems with this test:

- The 37 is a magic literal that apparently comes from nowhere.
- The test's name is utterly opaque about whether this is a test for the main-line case, a test for a common error condition, or a test that exists only because the coder was bored and thought it would be fun.

Combine these problems, and it quickly becomes next to impossible to fix the test a few months later when a change to the `User` class or the fixture file breaks it.

Naming tests is critical to being straightforward. Creating data locally and explicitly also helps. With most factory tools (see *Factories*, on page 109), default values are preset, so the description of an object created in the test can be limited to defining only the attributes that are important to the behavior being tested. Showing those attributes in the test is an important clue to the programmer's intent. Rewriting the preceding test with a little more information might result in this:

```
it "rounds total points to the nearest integer" do
  User.create(:points => 32.1)
  User.create(:points => 5.3)
  expect(User.all_total_points).to eq(37)
end
```

It's not poetry, but at the very least an interested reader now knows where that pesky 37 comes from and where the test fits in the grand scheme of things. The reader might then have a better chance of fixing the test if something breaks. The test is also more independent since it no longer relies on global fixtures—making it less likely to break.

Long tests or long setups tend to muddy the water and make it hard to identify the critical parts of the test. The same principles that guide refactoring